

CSCE 637 COMPLEXITY THEORY · MODULE 06 · LECTURE NOTES

Space

A different resource, with different theorems.

MODULE OUTCOMES

- Define L and NL and explain the input-tape convention.
- State Savitch's theorem and its consequence.
- Explain NL-completeness and log-space reductions.
- Explain Immerman–Szelepcsényi and why it is surprising.
- Explain the parallel connection and NC.

1 Logarithmic space

LOG SPACE MEANS LOG WORKING SPACE, WITH THE INPUT READ-ONLY

The input occupies n cells, so charging for it would make any sublinear space class empty. So the model is adjusted: a read-only input tape that is not charged, and a separate read-write work tape that is.

With that convention, L is the class of problems solvable in $O(\log n)$ work space — which is enough for a constant number of pointers into the input, a few counters, and nothing else. **You cannot store a list, a set, or a copy of anything.**

Which makes it a natural and practically meaningful class: a log-space algorithm is a streaming or pointer-chasing algorithm, and the restriction corresponds to a real engineering constraint (processing data larger than memory, or a constant-memory embedded routine).

And NL is the non-deterministic version, whose canonical problem is s-t connectivity in a directed graph — guess the path one vertex at a time, remembering only the current vertex and a step counter, which is log space. The certificate is the path and it is too long to store, which is why non-determinism helps here.

Class	What it means	Canonical problem
L	$O(\log n)$ work space, deterministic.	Undirected s-t connectivity — which Reingold proved in L in 2004, after decades open.
NL	$O(\log n)$ work space, non-deterministic.	Directed s-t connectivity, which is NL-complete.
PSPACE	Polynomial space.	TQBF (Module 05 §2).

Class	What it means	Canonical problem
$L \subseteq NL \subseteq P$	All inclusions known; none known to be strict.	The same situation as everywhere else in this course (Module 05 §2's landscape).

Undirected connectivity being in L was open for decades and was settled in 2004 by a randomness-free construction using expander graphs — **while the directed version is NL -complete and is not expected to be in L** . The gap between directed and undirected is a satisfyingly concrete form of a separation question, and the undirected result is one of the genuine recent successes in the field.

2 Savitch and Immerman–Szelepcsényi

SAVITCH: NON-DETERMINISTIC SPACE IS AT MOST SQUARED

$NSPACE(f)$ is contained in $SPACE(f^2)$. So $PSPACE = NSPACE$, and non-determinism buys nothing at all at polynomial space.

Contrast time, where the best known deterministic simulation of a non-deterministic machine is exponential — and whether it can be made polynomial is exactly P versus NP . So the two resources behave completely differently on the question that defines the field.

The proof is a recursive reachability test: can configuration A reach configuration B within 2^k steps? Try every possible midpoint M , and recursively ask whether A reaches M and M reaches B in 2^{k-1} steps.

Recursion depth k , with one configuration stored per level.

And space is reusable in a way that time is not, which is the whole reason the theorem holds: the recursion reuses the same memory cells across every branch it explores, and you cannot reuse elapsed time. That asymmetry between the two resources is the most important intuition in this module, and it explains both of its theorems.

IMMERMAN–SZELEPCSÉNYI: NL IS CLOSED UNDER COMPLEMENT

$NL = co-NL$. So directed s - t non-connectivity is also in NL — which is startling, because there is no obvious certificate for 't is not reachable from s' that fits in logarithmic space.

The technique is inductive counting: non-deterministically compute the *number* of vertices reachable within i steps, for each i , in a way that can be verified — and once you know the exact count, you can certify that a particular vertex is not among them, by exhibiting the others.

And this is exactly where the space analogue of the NP versus $co-NP$ question goes the other way (Module 05 §1): settled, and settled in the direction nobody would have guessed.

Which is a useful corrective: the analogy between time and space classes fails at the two most interesting points — Savitch (where space collapses non-determinism and time is not known to) and Immerman–Szelepcsényi (where space closes under complement and time is not known to) — so arguing from one resource to the other is unreliable, and intuitions about P versus NP drawn from space results should be distrusted.

3 Log-space reductions

LOG-SPACE REDUCTIONS, BECAUSE POLYNOMIAL ONES ARE TOO COARSE

Polynomial-time reductions are useless for distinguishing within P — every problem in P reduces to every other problem in P in polynomial time, by simply solving it. **So the notion collapses exactly where you wanted to use it.**

So the reduction must be weaker than the class it studies. Log-space reductions are the right notion for L, NL, and P-completeness — which is a general principle worth extracting: **a reduction is only informative if it is weaker than the distinction it is being used to draw.**

And they compose, which requires care. You cannot store the intermediate result — it may be polynomially long and you have logarithmic space — **so the composition recomputes each needed bit of it on demand**, re-running the first reduction from scratch for every bit the second one reads. **The proof is not trivial**, and it is a good illustration of what log space forces you to do.

P-completeness under log-space reductions is the important application: a P-complete problem is one that probably does not parallelise, which is §4 and is the practically useful part of the module.

4 Parallelism

NC IS THE PARALLEL-TRACTABLE CLASS, AND P-COMPLETENESS IS EVIDENCE AGAINST PARALLELISING

NC is the class of problems solvable in polylogarithmic time using polynomially many processors — 'efficiently parallelisable', formalised. **Equivalently, problems with polylogarithmic-depth, polynomial-size circuits** (Module 09).

NC is contained in P, and whether $NC = P$ is open — the parallel analogue of P versus NP, believed false for broadly similar reasons and with correspondingly similar barriers to proof.

So a P-complete problem is one that probably does not parallelise. The circuit value problem, linear programming, and computing the lexicographically first depth-first search ordering are all P-complete — and all three resist parallelisation in practice, which is reasonable circumstantial support.

Which is the practically useful content of this module, and it is the part relevant to CSCE 735: **if your problem is P-complete, the difficulty you are having parallelising it is structural rather than a failure of effort, and the response is a different algorithm or a relaxed requirement rather than more engineering.** 'Is it P-complete?' is worth asking before investing in parallelising anything, and checking against a catalogue takes an hour.

- **Savitch: non-determinism costs at most a squaring in space**, so $PSPACE = NPSpace$ — **and the time analogue is the open question that defines the field.**
- **Immerman–Szelepcsényi: $NL = co-NL$** , which has no known time analogue and is genuinely surprising.

- **So the time/space analogy fails exactly where it would be most useful**, and reasoning across it is unreliable (§2).
- **Log-space reductions are needed to distinguish inside P** — because a reduction must be weaker than what it classifies, which is a transferable principle.
- **And P-completeness is a practical warning about parallelisation** — the one result in this module you will actually use, and the reason space complexity deserves a module in a program with CSCE 735 in it.

Where to see this taught differently

Original material only goes so far — hearing the same idea in another voice is how it sticks. These are free, and they cover this module specifically.

Resource	What it covers for this module
Arora & Barak — chapter 4 (free draft)	Space classes, Savitch, and Immerman–Szelepcsényi, with both proofs.
Sipser — chapter 8	The same material more slowly, and the input-tape convention of §1 explained carefully.
Reingold — Undirected Connectivity in Log-Space (free)	§1's 2004 result, which is a genuine recent success and uses expander graphs in a surprising way.
Greenlaw, Hoover & Ruzzo — Limits to Parallel Computation (free)	§4's catalogue of P-complete problems, free in full — the reference to check before parallelising anything.

Exercises

Do these in code, not on paper, unless the exercise says otherwise. The module is not finished until they run.

- 1 **Write a log-space algorithm** for a problem of your choosing and verify the space bound.
- 2 **Explain why charging for the input tape would empty L.**
- 3 **Implement directed s-t connectivity non-deterministically** and confirm the space usage is logarithmic.
- 4 **Work through Savitch's recursion** and compute the space at each depth.
- 5 **Explain why the same argument fails for time.**
- 6 **Read the inductive counting argument** and summarise why it certifies non-reachability.
- 7 **Show that polynomial-time reductions cannot distinguish within P.**
- 8 **Explain how log-space reductions compose without storing the intermediate.**
- 9 **Look up three P-complete problems** and check whether you have tried to parallelise any of them.
- 10 **Take a problem you parallelised successfully** and find where it sits relative to NC.

Self-check

Answer these in writing, from memory, without looking back at the notes. If you cannot, you have read this module but not yet learned it — that is useful information, not a failure.

- 1 Why is the input tape not charged, and what would happen otherwise?
- 2 Define L and NL and give the canonical problem for each.
- 3 State Savitch's theorem and its consequence.
- 4 Why does space collapse non-determinism when time is not known to?
- 5 State Immerman–Szelepcsényi and say why it surprises.
- 6 Where does the time/space analogy fail, and what follows?
- 7 Why are log-space reductions needed, and what is the general principle?
- 8 Define NC and state the open question.
- 9 What does P -completeness tell you practically?